

IDNUK → SaaS

Turning the proven Sri Murugan & Co mandi platform into a self-service, multi-tenant SaaS —
without touching the current data model.

Self-service signup at idnuk.com

14-day free trial → subscription

Per-customer customization

Wave 1: 200 customers

99%+ uptime · secure · scalable

The short answer to your four questions

1 · Should we rewrite everything?

NO A rewrite is the #1 way startups die. Your PHP + React app already works in production. We make it tenant-aware with a **targeted refactor**, not a rebuild.

2 · Should we change hosting?

YES Shared cPanel cannot do programmatic provisioning, autoscaling or a 99%+ SLA. Move to **managed cloud (VPS + managed MySQL + Cloudflare)**.

3 · What multi-tenancy model?

Database-per-tenant. Each customer gets their own copy of your *current* schema. Your data model is reused verbatim — strongest isolation, easiest sell.

4 · How long & how much?

~13–16 weeks to general availability. Infra for 200 tenants ≈ **\$300–500/mo** (~₹25k–42k). Gross margin 80%+ at ₹999/tenant.

The rest of this deck details the architecture, hosting, security, costing, readiness gaps and a phase-by-phase rollout.

Where we are today

Current stack

- PHP (procedural) API · ~19 endpoint files · shared helpers/api.php
- React 18 + Vite SPA · two bundles (marketing + app)
- MySQL — **one database** (smanwefl_idnuk, ~35 tables/views)
- Namecheap shared cPanel · deploy via GitHub Actions (FTPS)
- Token sessions (sliding 8h), day-lock, audit log, backups, Sentry

What blocks SaaS today

- ✗ Single hardcoded DB — one business only
- ✗ No signup / tenant / subscription concept
- ✗ Shared hosting: no auto-provisioning, weak SLA, limited connections & scaling
- ✗ Secrets live in a committed config file
- ✗ No billing, trials, or plan/feature gating

What already helps us

- ✓ **migrateOnce()** runs per-DB → tenant migrations are basically free
- ✓ Permissions, audit log, day-lock, print templates, branding settings already exist

Three ways forward — and the recommendation

OPTION	WHAT IT IS	EFFORT / RISK	SCALES TO SELF-SERVICE?	
A · Manual clones	Copy the whole app+DB per customer by hand on a VPS	Low / Low	No — every customer is manual ops	STOP-GAP
B · DB-per-tenant SaaS	One control plane + dynamic per-tenant DB + signup/billing on managed cloud	Medium / Low-Med	Yes — fully automated self-service	RECOMMENDED
C · Full rewrite	Re-platform to microservices + shared-schema multi-tenant	Very High / High	Yes, eventually	AVOID NOW

Why not A

Fine for your first 2–3 customers, but 200 customers × manual setup, updates and backups is unworkable and error-prone.

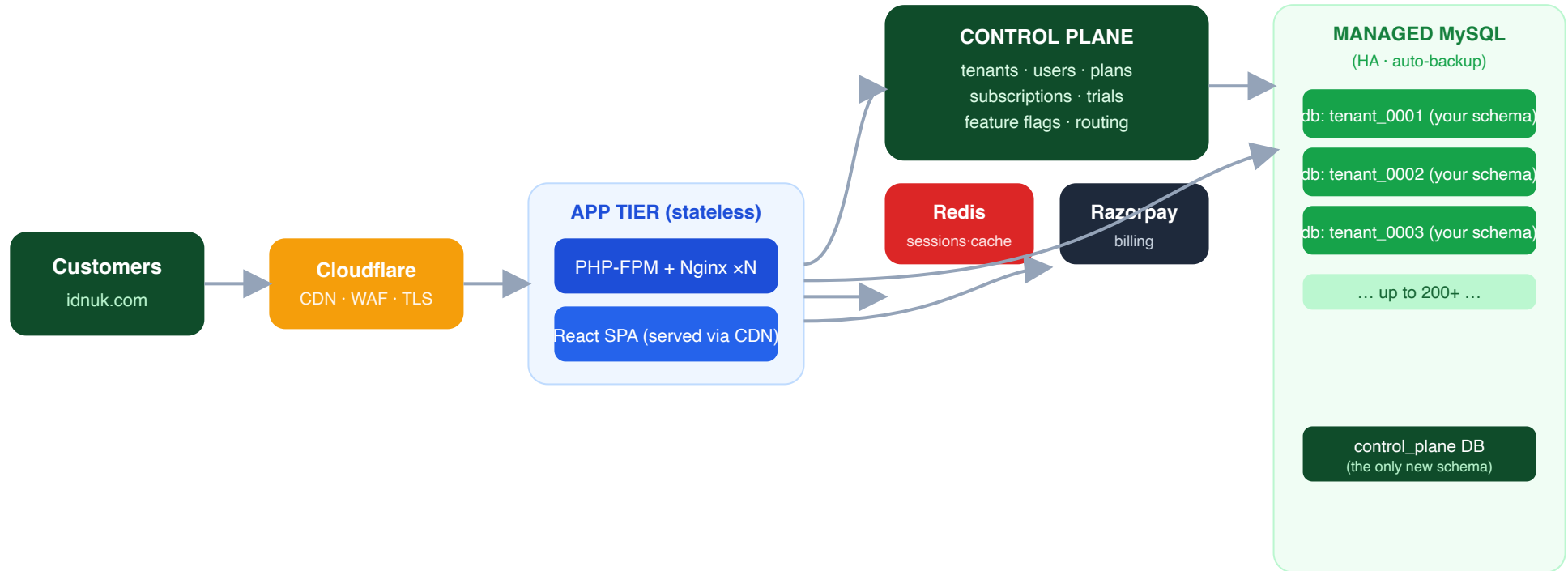
Why B wins

Preserves your data model, gives bank-grade tenant isolation, automates everything, and reaches thousands of tenants on modest infra.

Why not C

Months of cost for zero new customer value. Re-architect *incrementally* only where scale demands it.

Database-per-tenant, one control plane



Login resolves the tenant in the control plane → the app opens that tenant's database for the request. A query can never cross tenants, because the connection itself is scoped.

How this leaves your data model untouched

Today the app talks to one fixed database:

```
define('DB_NAME', 'smanwefl_idnuk');
```

We change **one thing** — resolve the DB name from the logged-in tenant instead of a constant:

```
$db = connectTenant($auth->tenant_db);  
// every existing query runs unchanged
```

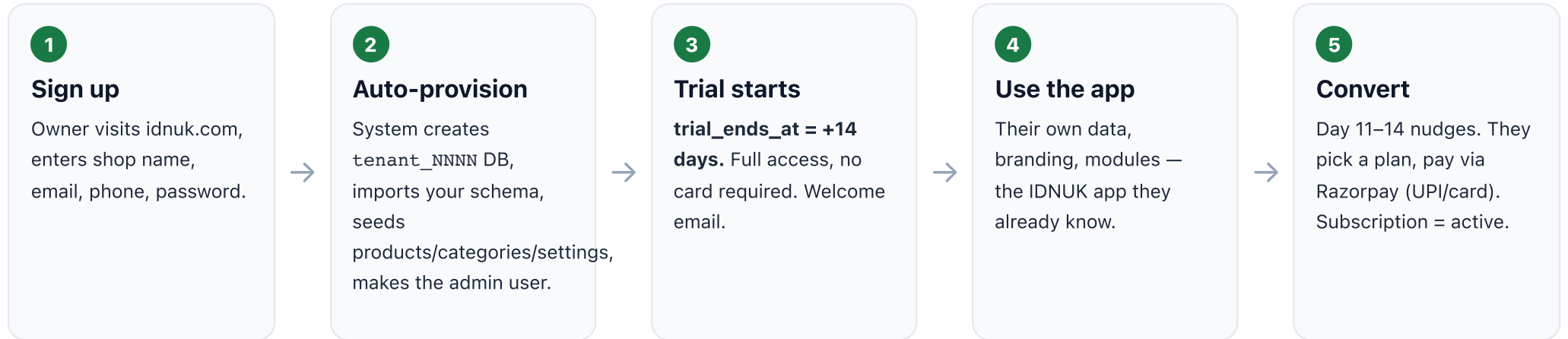
- ✓ Same tables, columns, views, indexes — a clone of today's schema
- ✓ Your pilot (Sri Murugan & Co) simply becomes **Tenant #1**
- ✓ **migrateOnce()** already self-migrates each DB on first hit after a deploy
→ schema changes roll out to all tenants automatically

What is genuinely new (small & separate)

- A **control-plane** database: tenants, subscriptions, trials, plans, feature_flags, platform_users
- A thin **tenant-router** in the auth layer (login → tenant → DB handle)
- A **provisioning** job (create DB, import schema, seed defaults)
- A **billing** module (Razorpay) + a marketing/signup site

None of this edits your business tables. It wraps around them.

Signup → 14-day trial → paid, with zero manual ops



If they don't pay

After day 14 the account moves to **read-only / locked** with a clear "Subscribe to continue" screen. Data is retained 30–60 days, then archived/purged per policy. No data is ever shown to another tenant.

Provisioning time

Target < **30 seconds** from "Create account" to "You're in." Schema import is a fixed template, so it's fast and identical for everyone.

Subscriptions & billing (Razorpay for India)

Why Razorpay

- UPI, cards, netbanking, wallets — what Indian SMBs actually use
- Native **Subscriptions API** for recurring auto-debit (UPI AutoPay / e-mandate)
- Webhooks drive subscription state; GST-compliant invoices built in

Subscription lifecycle

trialing	14 days, full access
active	paid, auto-renews
past_due	retry + grace, then lock
canceled	read-only → archive

⚠ Tax note — read this

Your **mandi produce** is GST-exempt — but **selling software (SaaS) is taxable at 18% GST**. You'll need GST registration for the SaaS business and must charge GST on subscriptions. Keep this entirely separate from the commission business. (Talk to your CA before launch.)

What we build

- ✓ Plan catalog (Starter / Pro)
- ✓ Trial countdown + paywall gate
- ✓ Razorpay checkout + webhook handler
- ✓ Invoices, receipts, billing history in-app
- ✓ Admin dashboard: MRR, trials, churn

Configurable, not forked — the golden SaaS rule

Every customer can look and behave differently — driven by **configuration**, never by separate copies of the code. One codebase serves all 200+ tenants.

Branding

- Logo, shop name (EN/Tamil)
- Print/bill templates
- Receipt headers, terms
- (already in your app)

Modules / Features

- Per-tenant **feature flags**: Yard, Market, Reports, Tally export...
- Plans unlock feature sets
- Sidebar shows only what's enabled

Business rules

- Commission %, coolie, sakku defaults
- Units, rounding, languages
- Working-date / day-lock policy

Discipline that keeps you sane at scale

If a customer needs something truly unique, it becomes a **configurable option for everyone**, not a private code branch. Per-customer code forks would make 200 customers impossible to maintain or update. Everything ships from one pipeline; customization is data.

Off shared cPanel → managed cloud

CAPABILITY	SHARED CPANEL (TODAY)	MANAGED CLOUD (TARGET)
Create DBs on the fly	✗ manual only	✓ programmatic provisioning
Scale app under load	✗ fixed, shared CPU	✓ add stateless app nodes
DB high-availability	✗ none	✓ managed MySQL w/ failover + PITR backups
Uptime SLA	~ best-effort	✓ 99.9% infra SLA achievable
Security (WAF/DDoS/secrets)	~ limited	✓ Cloudflare WAF + secrets manager
Cost at 200 tenants	cheap but a dead end	\$300–500/mo, headroom to thousands

Recommended providers (any one)

App: Hetzner / DigitalOcean / AWS Lightsail (Docker + Nginx + PHP-FPM). **DB:** DigitalOcean Managed MySQL or AWS RDS. **Edge:** Cloudflare. **Storage:** S3-compatible (backups, PDFs, logos). **Region:** India (Mumbai) for latency + data residency.

Keep what works

Same Git-based CI/CD (now building Docker images instead of FTPS), same Sentry, same backup logic — re-pointed at object storage and run centrally across tenants.

Comfortable at 200, headroom for thousands

Why 200 tenants is easy here

- A mandi is **low-throughput** (tens–hundreds of bills/day each). 200 of them is still light DB traffic.
- MySQL handles **thousands of schemas** on one managed instance — we watch table-cache & connections, add **ProxySQL** only if needed.
- **Stateless app nodes** + sessions in **Redis** → add nodes horizontally behind a load balancer in minutes.
- **Cloudflare CDN** serves the React app globally; API stays light.

Performance budget (the "premium feel")

First app load	< 2.0 s
Page/API response (p95)	< 400 ms
Signup → ready	< 30 s
Concurrent tenants	200 → 2,000 w/o redesign

We already added prefetch + skeletons + sliding sessions; the SaaS tier adds CDN + Redis + connection pooling to hold these numbers under load.

Built for handling other people's money

Tenant isolation

- DB-per-tenant = hard boundary
- Tenant resolved from the signed session, **never** from client input
- Per-tenant DB credentials

Platform security

- TLS everywhere · Cloudflare WAF + rate limits
- Secrets in a manager, **not** in git
- Brute-force lockout (have it), 2FA for admins
- Encrypted backups + restore drills

Compliance (India)

- **DPDP Act 2023**: consent, data-retention, deletion on request
- SaaS **18% GST** invoicing
- Audit log (have it) per tenant
- Pen test before GA

Immediate pre-launch must-dos (carried from ops hardening)

Rotate the DB password currently in `config/database.php` and move all secrets out of the repo · force HTTPS · enable centralized encrypted backups with tested restores · set up uptime + error alerting.

What "uptime" really costs — two tiers

TARGET	DOWNTIME BUDGET	WHAT IT REQUIRES	INFRA /MO
99% (your ask)	~7.2 h / month	Single well-run app node + managed MySQL + backups + alerting	~\$150–200
99.9% (recommended)	~43 min / month	2+ app nodes behind LB + managed MySQL with failover + Redis + Cloudflare	~\$350–500
99.99%	~4 min / month	Multi-AZ, read replicas, on-call rotation — premature today	\$1k+

Recommendation

Launch on the **99.9% tier** — the extra ~\$200/mo is trivial against 200 paying customers and protects your reputation while you're winning trust. It's the same architecture, just doubled app nodes + DB failover.

How we prove it

Uptime monitor on `/health` (have it), status page, Sentry error alerting, monthly backup-restore drill, documented runbook (have a start).

Gap analysis: today → SaaS-ready

AREA	HAVE	NEED TO BUILD	SIZE
Business app (bills, reports, ledger...)	✓ Production-proven	—	done
Per-DB migrations	✓ migrateOnce()	Central runner across all tenants	S
Auth / sessions	✓ Sliding tokens	Tenant-scoped JWT + Redis store	M
Multi-tenancy	✗	Control-plane DB + dynamic DB resolver	M
Signup & provisioning	✗	Marketing/signup site + auto-provision job	M
Billing	✗	Razorpay subscriptions + trial gating + invoices	M
Feature flags / plans	~ Permissions exist	Tenant-level flags + plan→feature map	S
Hosting / infra	✗ Shared cPanel	Cloud, Docker, CI/CD, CDN, managed DB, secrets	M
Observability / SLA	~ Sentry, health	Central logs, metrics, status page, on-call	S

S ≈ days · M ≈ 1–3 weeks. The business app — the hardest, riskiest part — is **already done**. The remaining work is the SaaS "wrapper".

Monthly infrastructure — 200 tenants

COMPONENT	99.9% TIER /MO
App nodes x2 (4 GB)	\$60–100
Managed MySQL (HA, 4 GB)	\$100–150
Redis (managed, small)	\$15–25
Load balancer	\$12–20
Object storage + backups	\$10–20
Cloudflare (Pro)	\$20
Email (transactional)	\$10–15
Monitoring + Sentry	\$0–30
Domain / TLS	~\$2
Total infra	\$240–480 /mo
Razorpay fees	~2% of revenue

≈ ₹20k–40k/month. Per-tenant cost ≈ \$1.2–2.4 (~₹100–200).

One-time build (engineering)

If you hire 1–2 engineers for ~3.5–4 months, budget the build as salary/contract for that window. The platform reuses your existing app, so it's **weeks, not a year**. Plus ~\$1–3k for a pre-launch pen test.

Other run costs

- Support tooling (helpdesk/WhatsApp): \$0–50/mo
- Razorpay payout/GST handled by your CA
- Scale to ~1,000 tenants ≈ 2–3× infra, not linear

The business case at 200 customers

Blended price

₹999/mo

Starter ₹799 · Pro ₹1,499

Monthly revenue @200

₹2.0L

≈ \$2,400 / mo

Gross margin

~83%

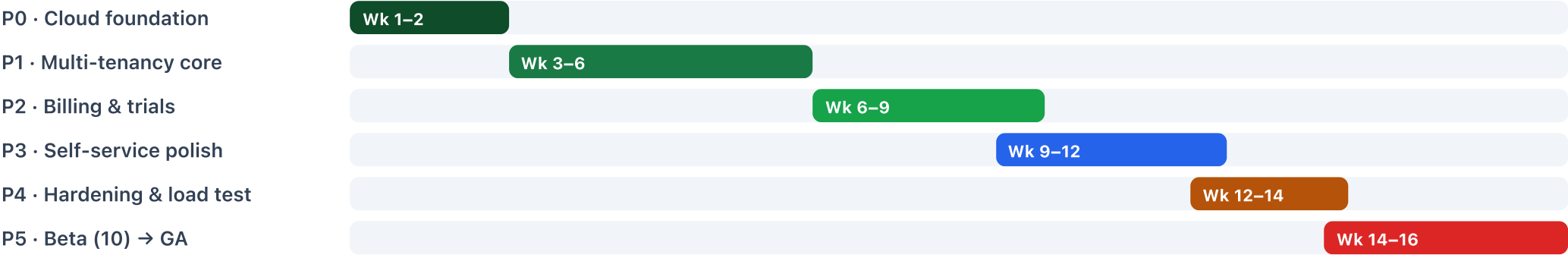
after infra + Razorpay

SCENARIO	CUSTOMERS	MRR	INFRA+FEES	GROSS PROFIT/MO
Beta	10	₹10k	₹16k	(invest)
Ramp	50	₹50k	₹22k	₹28k
Wave 1 target	200	₹2.0L	₹34k	₹1.66L
Scale	1,000	₹10L	₹1.2L	₹8.8L

Illustrative; refine price after beta. Even at 50 paying customers you're cash-flow positive on infra by a wide margin.

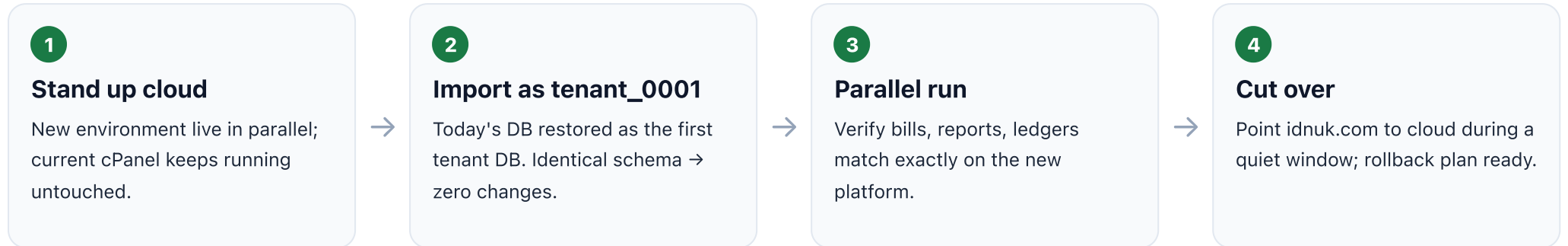
ROLLOUT TIMELINE

~13–16 weeks to general availability



PHASE	OUTCOME	DE-RISKS
P0	Pilot running on cloud + Docker + CI/CD + Cloudflare + backups	Hosting move (do it first)
P1	Control plane, dynamic DB, signup, auto-provisioning, Tenant #1 = your shop	The core bet
P2	Razorpay subscriptions, 14-day trial gate, invoices	Revenue
P3	Onboarding wizard, branding, feature flags, plan management	Self-service
P4	Pen test, 200-tenant load test, DR drill, runbook	SLA & trust
P5	10 real beta shops, fix, then open idnuk.com to all	Product-market fit

Sri Murugan & Co becomes Tenant #1 — safely



Risk to your live business: near-zero. The pilot's data and workflow are preserved byte-for-byte; the move is additive and reversible. Your shop is also the best possible reference customer for selling to the next 199.

What it takes to run this

To build (3.5–4 months)

- **1 senior full-stack** (PHP + React + cloud) — leads the platform
- **+0.5–1 engineer** for billing/frontend polish (can be part-time)
- **You** — product, pricing, first customers, domain expertise
- Fractional **DevOps** for the cloud setup (a few weeks)

To operate (ongoing)

- 1 engineer part-time for maintenance + features
- Support via WhatsApp/helpdesk (you + 1, scales with customers)
- Monthly: backup-restore drill, security patches, cost review
- CA for GST/invoicing on the SaaS entity

Build vs. hosted-billing trade-off

Using Razorpay (vs. building billing) and managed MySQL/Redis (vs. self-hosting) trades a little money for a lot of time and reliability. At your stage that trade is always worth it.

What could go wrong — and the guardrail

RISK	MITIGATION
Cross-tenant data leak (the #1 SaaS fear)	DB-per-tenant isolation + tenant from signed session only + automated isolation tests in CI
"Just one custom thing" → 200 code forks	Strict config-driven customization; features are flags, never branches
Hosting migration breaks the live shop	Parallel run + byte-for-byte verify + instant rollback; cPanel stays up until proven
Scope creep delays revenue	Ship the SaaS wrapper first; defer nice-to-haves to post-GA
Schema change must reach 200 DBs	migrateOnce() self-migrates per DB on first hit + a central runner with dry-run
Payment/GST mistakes	Razorpay handles compliance; CA reviews GST setup before launch
Key-person dependency	Docs/runbook, repo access for ≥2 people, managed services reduce ops surface

Recommended path

Go with Option B — database-per-tenant SaaS on managed cloud. No rewrite. Move hosting. Reuse your data model. ~13–16 weeks to GA.

- Launch on the **99.9%** reliability tier (~\$350–500/mo)
- Migrate your shop as **Tenant #1**, then beta with 10, then open to 200
- Razorpay for trials & subscriptions; configurable customization, never forks

Decisions I need from you

- Approve Option B + the hosting move?
- Target launch price (₹799 / ₹1,499 — adjust?)
- Hiring: 1 senior + part-timer, or an agency?
- Cloud preference (Hetzner/DO/AWS) & India region OK?
- Green-light P0 (cloud foundation) to start immediately?

Next: I can turn this into a phase-by-phase technical implementation plan (P0–P1), with the exact control-plane schema, the tenant-resolver code, and the provisioning script — all without touching your current business tables.